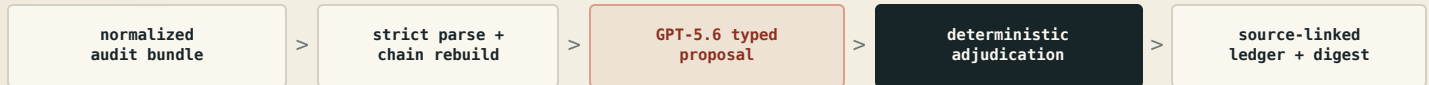


Codex Rule Ledger

Turns an already-normalized Codex launch-and-session bundle into a **source-linked evidence ledger** for reviewers deciding whether to accept, investigate, or block an agent-produced change. Most audit tools ask whether a rule passed. Rule Ledger first asks whether the supplied evidence makes any verdict admissible.

Dan Mercedes / Staff engineers, platform teams, and security reviewers

ARCHITECTURE IN ONE GLANCE



capture manifest + diff + session / recorded semantics by default / explicit local --live mode only

READ THE THREE AUTHORITY BOUNDARIES CORRECTLY

1 Input must earn admission

Strict schemas, byte and count bounds, content hashes, and allowlisted structure reject malformed or unknown material. Model-bound text is redacted before transmission.

SUPPORTED

2 GPT-5.6 proposes semantics

Live mode maps complete instruction lines into typed, source-linked proposals. It uses an environment-only key, store: false, no tools, and zero automatic retries.

CONTRADICTED

3 TypeScript owns the verdict

Deterministic code validates digests, coverage, exact quotes, rule form, and evidence sufficiency. It alone assigns results and the canonical SHA-256 export.

NOT_EVIDENCED

NOT_APPLICABLE

Subjective language is declined. Ambiguity routes to human review. NOT_EVIDENCED is neither failure nor compliance.

THE HARDEST DESIGN CALLS

01 Admissibility before pass or fail

A diff shows what changed, not which layered instructions governed the captured run or whether its evidence supports a completion claim.

02 Missing evidence gets its own state

Treating absence as compliance admits an unverified "done." Treating it as failure rejects work that may have happened outside the supplied capture.

03 Retries change what failure means

Case 001 links failed typecheck plus later completion to a contradiction. Case 002 demonstrates why an exact successful retry must prevent that false contradiction.

04 Public judging stays recorded and keyless

Two disclosed synthetic cases cross the same typed analyzer seam as live mode without visitor input, persistence, a public model route, or API spend.

WHAT IS PROVEN

- 77/77 unit and contract tests across seven files plus 5/5 Chromium acceptance flows.
- 42 witnessed vertical RED-to-GREEN slices from living specs through release.
- One GPT-5.6 proof call through the committed CLI: 1,323 input / 820 output tokens, zero retries, \$0.031215.
- Keyless production: root 200, read-only audit 200, mutation 405, zero configured Vercel project environment variables.

CI requires lint / typecheck / unit / deterministic boundary / build / E2E / Gitleaks / dependency review.

HONEST V0.2 SCOPE

- BUILT** Browser explorer, read-only API, local Audit Bundle CLI, recorded and live analyzer adapters, canonical export.
- PUBLIC** Exactly two repository-owned synthetic recorded cases. No login, key, upload, persistence, or public GPT endpoint.
- LIMIT** Reports supplied evidence. Does not certify compliance, authenticate a trace, establish trusted time, normalize raw sessions, or execute captured commands.

JUDGE IT IN 60 SECONDS

1 / Inspect Case 001

Open the contradicted typecheck rule and follow its source, failed command, and later completion event.

2 / Switch to Case 002

See the distinct retry-recovery topology and why exact success prevents a false contradiction.

3 / Export the ledger

Download canonical JSON whose filename and metadata bind the SHA-256 digest.

No account. No API key. No upload.

Local CLI:

```
npm run --silent audit -- \
  --bundle fixtures/synthetic-retry-recovery-v1 \
  --out ledger.json
```